



PDF Download
SC.2018.00040.pdf
15 February 2026
Total Citations: 6
Total Downloads: 91



Published: 11 November 2018

Citation in BibTeX format

SC18: The International Conference
for High Performance Computing,
Networking, Storage and Analysis
November 11 - 16, 2018
Texas, Dallas

Conference Sponsors:
SIGHPC

 Latest updates: <https://dl.acm.org/doi/10.1109/SC.2018.00040>

RESEARCH-ARTICLE

A reference architecture for datacenter scheduling: design, validation, and experiments

GEORGIOS ANDREADIS, Delft University of Technology, Delft, Zuid-Holland, Netherlands

LAURENS VERSLUIS, Free University Amsterdam, Amsterdam, Noord-Holland, Netherlands

FABIAN MASTENBROEK, Delft University of Technology, Delft, Zuid-Holland, Netherlands

A. IOSUP, Delft University of Technology, Delft, Zuid-Holland, Netherlands

Open Access Support provided by:

Delft University of Technology

Free University Amsterdam

A Reference Architecture for Datacenter Scheduling: Design, Validation, and Experiments

Georgios Andreadis*, Laurens Versluis[†], Fabian Mastenbroek*, and Alexandru Iosup*[‡]

*Delft University of Technology, [†]Vrije Universiteit Amsterdam, the Netherlands

{G.Andreadis, L.Versluis, F.Mastenbroek, A.Iosup}@atlargo-research.com

Abstract—Datacenters act as cloud-infrastructure to stakeholders across industry, government, and academia. To meet growing demand yet operate efficiently, datacenter operators employ increasingly more sophisticated scheduling systems, mechanisms, and policies. Although many scheduling techniques already exist, relatively little research has gone into the abstraction of the scheduling process itself, hampering design, tuning, and comparison of existing techniques. In this work, we propose a reference architecture for datacenter schedulers. The architecture follows five design principles: components with clearly distinct responsibilities, grouping of related components where possible, separation of mechanism from policy, scheduling as complex workflow, and hierarchical multi-scheduler structure. To demonstrate the validity of the reference architecture, we map to it state-of-the-art datacenter schedulers. We find scheduler-stages are commonly underspecified in peer-reviewed publications. Through trace-based simulation and real-world experiments, we show underspecification of scheduler-stages can lead to significant variations in performance.

Index Terms—Reference Architecture, Datacenter, Scheduling.

I. INTRODUCTION

Datacenter infrastructure is important for the digital society. Stakeholders across industry, government, and academia employ diverse cloud services hosted by datacenter infrastructure, and expect services to be reliable, high speed, and low cost [1], [2]. In turn, datacenter operators must maintain efficient operation at unprecedented scale [3]. Key to datacenter operation is its *scheduler*, which takes on behalf of users and datacenter-engineers decisions about workload and resources [4], [5]. To keep up with growing demand and increasing complexity, architects of datacenter schedulers must address complex challenges in distributed systems [5]–[8], software engineering [9]–[11], and performance engineering [12]–[14]. The entry level is high. Although *reference architectures* [15]–[17] help with complexity and entry-level problems, such *conceptual models* currently do not exist for this field. In this work, we propose a reference architecture for datacenter scheduling, and use it to analyze academia- and industry-designed schedulers.

A conceptual model capturing the entire process of datacenter scheduling could be beneficial to understand how to design, build, and control such complex systems [18]. It could facilitate an understanding of how scheduling works in datacenter ecosystems, including the complex and dynamic interplay between the tens of algorithms (policies) currently at work in real deployments. The model and associated tools, such as community-curated experimental platforms, could help

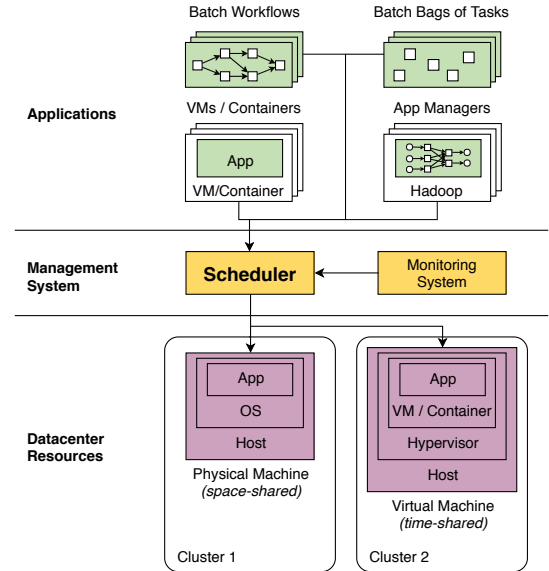


Fig. 1: Generic system model for datacenter operation. In this work, we create a reference architecture for the “Scheduler”.

analyze and compare schedulers in detail, and thus counter the problem that many new schedulers and policies (algorithms) appear each year, but few are well-understood or adopted in practice [19]. By summarizing patterns and best-practices, the model could help standardize scheduler construction.

The lack of a reference architecture, or a community-wide underlying conceptual model, can be costly. Conceptually, this could mean overlooking important components (e.g., replication, see results in Section IV-E). In practice, without a reference architecture, even capable engineering teams could tinker, leading to architectural and deployment issues, and to challenges in rolling back new projects into the main design.

How to design a reference architecture for datacenter scheduling? is the fundamental research question addressed in this work. The answer is non-trivial. The conceptual model must exceed the limits of current ‘black-box’ models, because the diversity of actions a scheduler needs to perform is not properly represented by a monolithic and opaque component. It must also avoid including too much detail, and thus must trade-off the detail of representation vs. the difficulty of describing the scheduling process; otherwise, the model would become a poor instrument for science, design, and engineering. The model must support the main workload, resource, and resource management and scheduling elements (see Figure 1, with terms defined in Section II).

We envision a *workflow-based scheduling model*, providing the overall view and a level of detail below it, and the key data and control flows between its workflow-stages. Inspired by the work of Schopf [20], who proposed an 11-step abstraction for grid scheduling, we focus on a conceptual model supporting brokered and hierarchical schedulers, but also the diverse set of scheduling operations in datacenters that appears in peer-reviewed research and in practice. The model supports common datacenter workloads, from workflows and bags-of-tasks to VMs and container-based hosting of black-box applications, and encompasses common actions in datacenter scheduling, ranging from filtering resources available to the user to task migration.

To validate the model in terms of validity and usefulness (defined in Section III-A), we focus on the question of how *underspecified* existing schedulers are relative to the conceptual model, that is, how peer-reviewed specifications about schedulers lack important details for understanding the scheduling-stages. Underspecification is the consequence of a trade-off between the benefits and the costs of documentation. Advocates for reproducibility already argue for the importance of specifying designs and experimental setups, both as a principle and in practice [21].

The model we envision aligns well with the general benefits of conceptual models, but also enables numerous pragmatic possibilities, two of which we explore in this work. Through *static analysis*, we identify the schedulers with comprehensive detail or underspecification, the scheduling-stages that are least and most underspecified, whether the academia or the industry underspecify the most, whether traditional or recent systems are the most underspecified, etc. We further use *dynamic analysis* to understand the effect of underspecification on the performance and operation of datacenter schedulers at run-time. To this end, we conduct simulation-based and real-world experiments, using as input representative traces collected from real-world environments [22].

Overall, the main contribution of this work is three-fold:

- 1) We design a reference architecture for datacenter schedulers (Section III).
- 2) We show how state-of-the-art schedulers can be mapped to the reference architecture, and conduct a static analysis of scheduler specifications (Section IV).
- 3) Through dynamic analysis, we assess the impact of underspecification on the performance and operation of datacenter schedulers (Section V).

II. GENERIC MODEL OF DATACENTER OPERATION

We use in this work a generic model of datacenter scheduling that is already widely used in the academia and deployed by the industry [4], [5], [23]. Figure 1 depicts this model.

A. Workload

The workload is comprised of a *stream of jobs* of various morphologies, that is, shape, structure, scale, etc. We assume all jobs fit the morphology of *workflows* [24]–[26]: each job consists of a set of one or several *tasks*, with precedence

constraints between tasks determining the order of execution. Workflows are common in practice, from science [27, p.137–146] [28] to engineering [29], from business processes [30] to applications built with serverless cloud services.

This model, of *workloads of workflows* encompasses: 1) *Bag-of-Tasks (BoT)*–jobs comprised of (many) tasks without interdependencies, 2) *hosted jobs*, such as business-critical workloads running inside hosts (e.g., VMs or containers), whose hosts correspond to the long-running tasks of a trivial workflow, 3) *managed jobs*, such as Hadoop- or Spark-based big data applications, or elastic jobs coordinated by an autoscaler [31], whose tasks (e.g., Map and Reduce for Hadoop) are coordinated by a long-running master-task in the workflow.

Workflows can form *hierarchical workloads*, that is, tasks in higher-level workflows can themselves take the morphology of a workflow; this process can be recursive.

The user can specify *requirements for each job*, which schedulers must fulfill. To become valid hosts for the job, resources must match *resource-requirements*, such as hardware architecture, OS, etc. Requirements can also include more diverse *Service Level Agreements (SLAs)* [32], such as the cost of leasing the resource [33], [34], elasticity [35], licenses and other codified legal aspects, etc.

B. Datacenter Resources

In our model, datacenters are comprised of physical and virtual resources. We consider a generic resource model: physical resources, such as nodes in a datacenter rack, and virtual resources, such as VMs or containers, are typically grouped into physical or virtual *clusters* [5].

We model in this work resources only through abstract operation, a model commonly employed in analytical and simulation-based studies [36], [37]: we model (1) *processing resources* with a generic model of production, e.g., FLOPS or MIPS, (2) *memory and storage resources* with a generic model of size and bandwidth but no latency, etc.

C. Resource Management and Scheduling

The *user* submits a workload to the datacenter, through a *central submission site*. Serving the central submission site is the *scheduler*, which follows a common but complex operational model [4], [5]: the scheduler matches streaming workloads to existing resources, selects and allocates resources for jobs, conducts setup actions for jobs and tasks, executes the workload, and also manages the workload through its life-cycle [5]. If an application requires more resources during its execution, or can release some of the resources it has been allocated, the scheduler can use an autoscaler to conduct these dynamic, elasticity-related *provisioning operations*. Autoscalers can be workflow-agnostic [12] or workflow-specific [14].

While executing the workload, the scheduler may preempt, recover, or migrate tasks, based on various events. Events may include performance variability [38], (correlated) resource failure [39], evolving leasing costs [40], etc. Events are provided primarily by the “System monitor” component (Figure 1), which continuously gathers resource-state information.

Although schedulers can operate as central monoliths, they can also be divided into smaller, cooperating instances. This can be the case in *hierarchical scheduler* models, where the scheduler operates as a hierarchy that typically matches the datacenter topology [41]. For example, local schedulers (*Local Resource Manager (LRM)*) could manage each a single cluster, with a datacenter-level scheduler distributing workload between LRMs. Custom setups that deviate from the standard scheduling flow can be facilitated by a *broker* [42], which is a component that can negotiate resources for workloads directly, potentially across different providers (schedulers).

The scheduler takes decisions guided by *policies*, separated by design from the mechanism that executes the decisions. Many policies already exist for scheduling in datacenters and clouds [43], [44]. For example, to rank the tasks eligible for execution, schedulers could use Shortest-Remaining-Time-First (SRTF), where tasks with shorter (estimated) execution time are given a higher priority over tasks with longer times. As another example, to take decisions related to resource allocation, schedulers could use the *First-Fit* policy, which selects the first resource with enough capacity to fit the task.

We explicitly do not consider here the scheduling processes occurring at user- and/or framework-level, that is, dynamically managing the computation and communication processes occurring inside the application. For example, the general model supports the workflows that users would build in Spark, but not the fine-grained, framework-level operations, such as RDD checkpointing, migration, and broadcasting.

III. DESIGN OF A REFERENCE ARCHITECTURE FOR DATACENTER SCHEDULING

In this section we design the reference architecture.

A. Main Requirements

Every design is created to fulfill a set of requirements, corresponding to the key problems identified by the design stakeholders. Key stakeholders to this design are researchers in the field of resource management and scheduling, datacenter operators, and students interested in the field. In this work, we consider two main requirements, validity and usefulness, which correspond to the complete set of stakeholders.

R1. Validity is the property of the proposed model to accurately represent the field of datacenter scheduling. For this, the model needs to cover the state-of-the-art, and in particular instances of datacenter scheduling from both real-world practice and emerging concepts from the academia. Although verifying this requirement is fundamentally a subjective task, mapping existing schedulers to the model embodied by the reference architecture (discussed in Section IV) and peer-review both give evidence the model is relevant.

R2. Usefulness gives the reference architecture a real-world purpose, which motivates its creation. As with validity, this property cannot be measured exactly, but the empirical results presented in Sections IV and V, combined with the future uses of this architecture we envision in Section VII, give evidence of usefulness for the reference architecture.

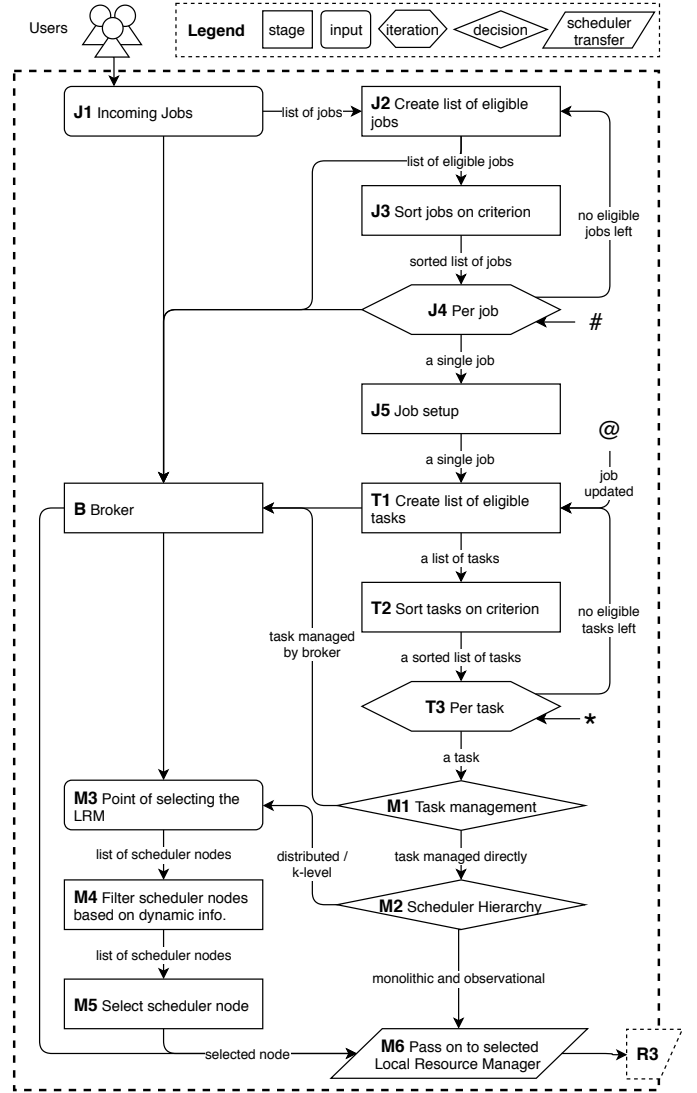


Fig. 2: Reference architecture for datacenter scheduling. Focus on the *global* scheduler. (Continued in Figure 3: the *, @, and # symbols are jump labels to Figure 3.)

B. Leading Design Principles

We have derived five leading principles for the design of the reference architecture for datacenter scheduling, which we use to design the reference architecture in Sections III-C and III-D:

P1. Components with Clearly Distinct Responsibilities: Essential to our design is the decomposition of the datacenter-scheduling process into individual components. Derived from the common practice of designing software architectures [45], each system component has a set of responsibilities, defined boundaries of those responsibilities, and a set of interfaces which define its services to other components. The set of components identified in the design-process gives choices when designing specific schedulers—not all components are used by every datacenter-scheduler deployed in practice.

P2. Grouping of Related Components: Corresponding to best-practices for packaging components [45], related components can be grouped according to their responsibility. De-

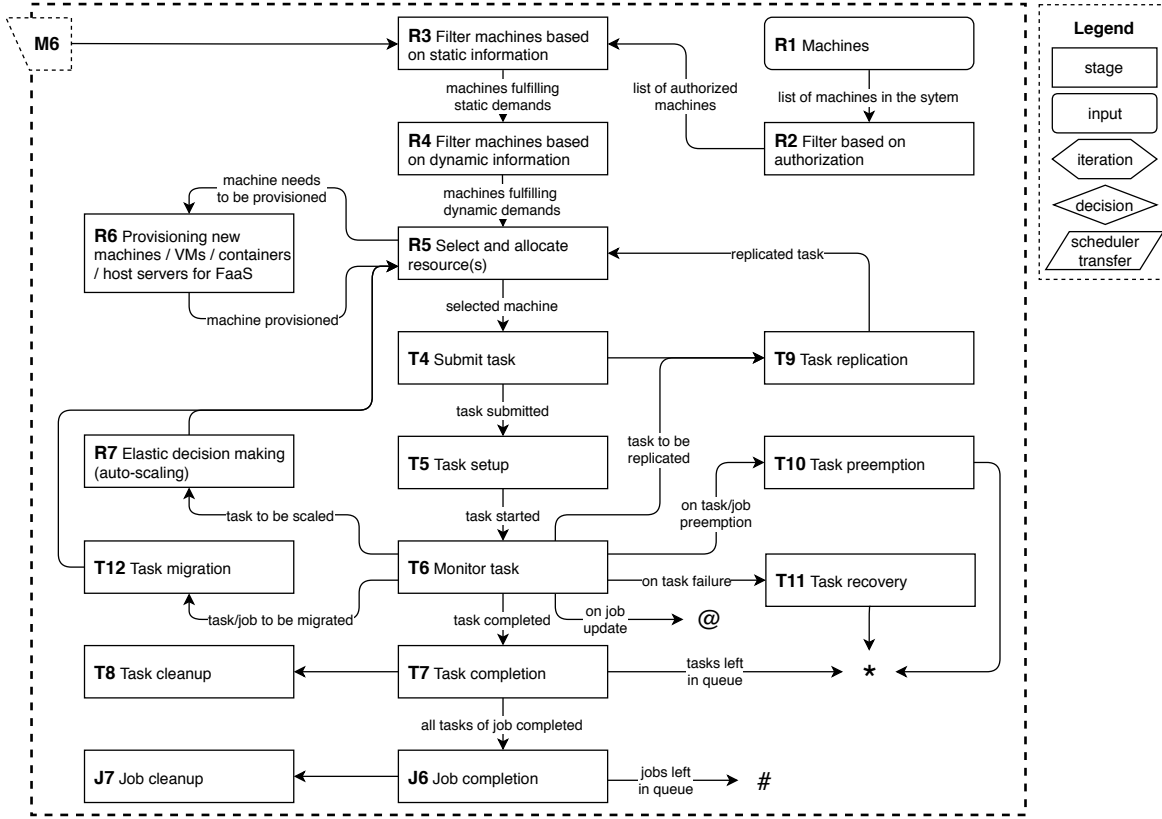


Fig. 3: Reference architecture for datacenter scheduling. Focus on the *local* scheduler. (Continued from Figure 2.)

pending on stakeholder and/or viewpoint, the components of the same architecture can lead to different grouping schemes.

P3. Separation of Mechanism from Policy: The definition of abstract components separates mechanism from policy. To provide the necessary level of abstraction, the reference architecture should not mandate specific policies—this model is *qualitative*, as defined in [45].

P4. Scheduling as Complex Workflow, Matching the System Model (see model defined in Section II-C): Schedulers act as workflows, with well-defined data and control flows between the workflow components. Although we consider in this work reference data- and control-flows, we can expect that in practice different flows will emerge, related to the physical topology and the operational goals of each datacenter.

P5. Hierarchical Scheduler with Shared Control: Derived from the system model, the scheduler is structured as a hierarchy, subject to the central control of the datacenter operator, and to partial control by or on behalf of individual datacenter-users. Both the hierarchy and the model for shared control are expressed through policies, but the latter policies may be changed more frequently in practice.

Concerning the accuracy of the model, we have mapped diverse and complex schedulers to the reference architecture built with these principles (next section, independent two-reviewer process). In Appendix B, we make in turn assumptions about various aspects of expertise and of conceptual complexity of components, and analyze their impact on mapping. We find no evidence that the model would be inaccurate.

C. Overview of the Reference Architecture

We present in this section the *overall design* and the *grouping* of components addressing the major responsibilities of datacenter scheduling.

Following principle *P1*, the reference architecture for datacenter scheduling depicted in Figures 2 and 3 models the scheduler as a set of components (term used interchangeably in this work with *stages*), each with specified inputs, outputs, and side-effects, the combination of which define its function. Each stage can be equipped with its own policy (*P3*), which is often a simple, heuristic algorithm. Section III-D details this.

Following *P4*, the scheduler forms, through control and data flows, a workflow that matches the main functions defined in the system model (Section II-C): execution starts at the central submission site, and progresses until job and task completion and cleanup stages. The execution flow of the scheduler (one *scheduler iteration*) may be started periodically, on events such as job arrival or completion, or even manually. In the scheduling workflow, control is transferred between stages through outputs, which are depicted as text on the interconnections between components in Figures 2 and 3 (e.g., in Figure 2, stage J1 transfers data, a “list of jobs”, to stage J2), or through events such as the completion of a stage, which are indicated as interconnections without description in the same figures (e.g., in Figure 3, the completion of stage T7 triggers the start of stage T8).

Following *P2*, we identify four major responsibilities:

- 1) *Job processing* (J): activities concerning the selection of jobs and the job life-cycle, such as job setup and cleanup,
- 2) *Task processing* (T): stages of the task life-cycle, including more sophisticated stages for task migration, preemption, and replication,
- 3) *Scheduler management* (M): stages facilitating scheduling hierarchies and (cloud) brokers, and
- 4) *Resource management* (R): stages related to provisioning and allocating resources.

Following *P2* and *P5*, we define another grouping of stages to emphasize the scheduler hierarchy: Figure 2 depicts the stages executed by a *global* scheduler (datacenter-level), whereas Figure 3 visualizes stages predominantly executed at a subordinate, *local* scheduler (cluster-level). As for control and data flow, this categorization only serves as a suggestion and may be deviated from for different schedulers (e.g., a monolith scheduler may execute all stages on the same machine). The *Broker* component is represented as a stub; the broker itself can employ more complex sub-systems, in a design that remains external to this architecture.

D. Details of the Reference Architecture

In this section, we present the design of the individual stages addressing the major responsibilities of datacenter scheduling.

To design the reference architecture at this level of detail, we focus each stage on a specific responsibility, and select for it one of the five *operational modes*: processing, input, iteration, decision, and transfer. For example, *iterative stages*, such as J4 and T3 in Figure 2, repeat the remainder of the flow they are connected to, for each element of input that they receive. However, the execution of these iterations is not restricted to sequential execution. Iterative stages may launch multiple, parallel threads of execution, that is, J4 may trigger multiple jobs to be processed in parallel, each starting with J5 and going through the rest of the process, independently. (This requires parallel-code constructs and consistency protocols [13].)

This reference architecture also permits stages to be either *stateless* or *stateful*. Stateful stages persist state across invocations and take historical information into account. We further differentiate between two types of *expected outputs* that these stages can produce: *outputs as a function of input* (including historical data for stateful stages), such as the set outputs of task filtering sorting stages, and *side-effects*, such as the actions of job setup and cleanup stages.

A full description of the components appears in our technical report [46]. We focus now four stages that feature prominently in the experiments we conduct in Section V:

- 1) *Task Filtering* (T1 in Figure 2): this stage filters the list of tasks provided as input, based on a filter-policy, for example, a policy that allows tasks to pass only if they are not depending on tasks that are yet to finish.
- 2) *Task Sorting* (T2 in Figure 2): any policy to order tasks, to improve the throughput and latency of tasks, e.g., SRTF, FIFO, and even uniformly random (RANDOM).
- 3) *Dynamic Resource Filtering* (R4 in Figure 3): a policy-based filter yielding a list of resources with sufficient

TABLE I: Overview of the mapping-based validation.

Section(s)	Analysis	Scheduler(s)	Aggregation
IV-B	Detailed mapping	Borg	None (all stages)
IV-C	Mapping overview	All	By stage-group
IV-D, IV-E	Head-to-head comp.	All	By feature

resource-capacities, based on fixed or dynamic requirements, and on predicted or monitored information about processing unit availability, memory occupancy, etc.

- 4) *Resource Selection* (R5 in Figure 3): in this stage, the selected task is matched with a (set of) resource(s), using policies such as First-Fit, Worst-Fit, and Best-Fit.

IV. VALIDATION THROUGH MAPPING OF SCHEDULERS

Reference architectures are most useful when they accurately depict real-world instances. In this section, we validate the reference architecture by mapping to it fourteen diverse real-world, well-known, and state-of-the-art schedulers, in the three experiments using static analysis summarized in Table I. We present in this section only a relevant and representative selection of mapping results (the complete set is in [46]).

The schedulers are selected based on two criteria: (1) the scheduling core (e.g., policies) have been analyzed and presented for the systems community, and (2) the peer-reviewed material is highly cited or used in a company running large datacenters. The first, detailed mapping of one scheduler to the reference architecture, is intended to exemplify the mapping process (see Section IV-A).

The second mapping, in Section IV-C, tests the validity of the reference architecture (requirement *R1*), by conducting the detailed mapping of each scheduler considered in this work, then aggregating the results into a tabular overview. We define *underspecification* for a scheduler as the degree of specification, as presented in peer-reviewed publications, of the scheduling stages included in the reference architecture, and of the policies used by the scheduler. *Which (groups of) stages are the most, or the least, underspecified? Which schedulers do the most, or the least, in specifying their stages?*

Fulfilling requirement *R2*, the mappings in Sections IV-D and IV-E help understand how schedulers map to the reference architecture, by asking relevant questions that divide schedulers into feature-based classes: *Are schedulers designed in the academia better specified than industry-designed schedulers? Are newer designs better specified than older designs?*

A. The Mapping Process

The mapping process proceeds as follows. Two reviewers have split the mapping work, conducting parallel mapping processes except for a small overlapping set of schedulers, including Borg [5], used for calibration; whenever the results for the overlapping set were found to be different across reviewers, they have discussed the reasons, reconciled their views, and adjusted their mapping process accordingly. Both experts and non-experts can apply this process (see Appendix B).

For each scheduler, we start the mapping process from the relevant peer-reviewed publications summarized in Table II.

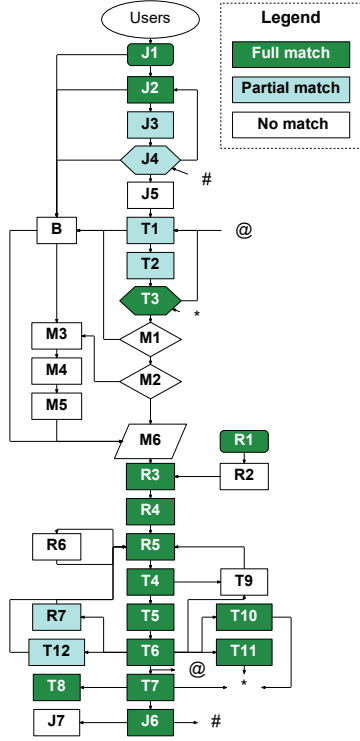


Fig. 4: Borg (Google) mapped to the reference architecture.

We further consider other publications, and *source-code inspection* when implementation details of the schedulers are accessible and well documented, for example through GitHub.

Stages for which the reviewers can find a credible description of both what the stage does, and how (both mechanism and policy), are labeled as “*full match*”. Stages for which the reviewers can find only one of the what and how, or for which the description of the how does not credibly match the what, are labeled as “*partial match*”. The remaining stages are labeled “*no match*”. A special case occurs when the scheduler has both a peer-reviewed publication and publicly accessing source-code: we do not make a distinction between stages that are not mentioned in the publication and stages that are not present in the actual scheduler implementation; in particular, the former results in a “no match” label for that stage.

B. Exemplary Mapping of Borg (Google)

To illustrate how the reference architecture allows studying state-of-the-art systems, in this section we map to the reference architecture Google’s Borg scheduler [5], and depict the result in Figure 4. Although scheduling at Google has evolved, Borg remains the seed of the concepts seen later in Omega and in Kubernetes [5], and the learnings of the Omega project have been reabsorbed into the production-scheduler Borg [13].

1) *Workload*: Similarly to the proposed reference architecture, users provide the Borg system with a workload in the form of jobs, structured as bags of tasks. The Borg system also allows jobs to be prioritized, both coarse-grained (i.e., production or non-production) and more fine-grained. This matches the ‘Sort jobs on criterion’ stage (J3), but we

label this stage “partial match” because, see [5, §2.5], (i) the production tasks are allocated a majority of the resources and cannot preempt each other, (ii) the details for fine-grained priority indicate the need for careful manual tuning, and (iii) no details are provided on whether the tasks are actually sorted next to being ranked. Jobs in Borg can also have constraints on, e.g., processors or OSs they can run on. This matches the ‘Filter machines based on static information’ stage (R3); here, enough information exists to label this stage a “full match”.

2) *Resource Organization*: Jobs generally run in one “cell”, that is, a set of machines that is considered one unit. One or more cells run in a physical cluster, and one or more clusters form a datacenter. Cells are managed by a central scheduling entity (“Borgmaster”, global) and machines by a local agent (“Borglet”, local). In this hierarchical model, the global scheduler selects, at the ‘Scheduler Hierarchy’ decision point of the architecture (M2), the lower-level (local) scheduler nodes to take over each job. (For users having access to multiple Borgmasters, we can envision another hierarchical level, whose use is possibly facilitated by the broker (B), but this does not appear in the Borg description [5].)

Corresponding to jobs based on VMs or containers in our model, resources can be reserved through “allocs” (single machine) and “alloc sets” (multiple machines). This corresponds to the ‘Select and allocate resource(s)’ stage (R5).

3) *Task Life-Cycle and Operations*: All of the T-stages apply to the Borg system. For T6, “Borglets” assume the role of monitoring tasks, providing information about their machines to higher-level schedulers (and to human engineers).

The Borg system supports policy-based task replication (T9), preemption (T10), recovery (T11), and for some jobs also migration (T12). For example, Borg uses for resource reclamation a combination of ‘Task monitoring’ (T6), ‘Task preemption’ (T10), and ‘Elastic decision making’ (R7, but the autoscaling mechanism lacks detail in [5, §5.5]): to ensure efficient use of all resources, the Borgmaster monitors per-task usage, real-time, and marks unused resources as free for lower-priority jobs (such as batch jobs). When a task needs again more resources, lower-priority jobs are preempted.

4) *Scheduling Algorithm*: At Borgmaster-level, the scheduling process consists of two main steps. First, during feasibility checking, the process selects machines fitting the task, by combining static and dynamic information, and thus by using both machine-filtering stages (R3 and R4).

Second, corresponding to the ‘Select and allocation resource(s)’ stage (R5), the process gives each machine a score based on a combination of metrics, and then uses a policy to select a machine (e.g., First- and Best-Fit).

C. Mapping Overview

We have repeated for all schedulers considered in this work the process described in Section IV-A and outlined for Borg in Section IV-B. Table II summarizes the results of the mapping, including statistics for the *groups* of stages for each of the four major responsibilities (see Section III-C), and detailed information for four selected stages (the focus

TABLE II: Summary of schedulers mapped to the reference architecture. Numbers express matching per group, as mean percentage over the scheduling tasks of the group, with 100%/50%/0% for a full/partial/no match per task (see text).

Scheduler	Feat.*	Group of Components				Selected Components*				
		J	T	M	R	T1	T2	R4	R5	
Condor [6]	A,O,M	64	88	28	71	✓	✓	✓	✓	
Mesos [10]	A,N,M	14	46	0	71			✓	✓	
Borg [5]	I,N,M	57	79	0	64	~	~	✓	✓	
Fuxi [47]	I,N,M	57	54	0	50	~		✓	~	
Autopilot [48]	I,O,S	36	46	0	29				~	
Sparrow [4]	A,N,S	21	33	57	0		~		~	
Pegasus [25]	A,N,M	71	63	0	36	✓		✓	✓	
Quincy [49]	I,O,S	43	50	0	28				✓	
ICENI [50]	A,O,S	50	33	0	57	~			✓	
Firmament [51]	A,N,S	14	29	28	57	~	✓	✓	✓	
Apollo [7]	I,N,S	85	75	100	57	✓	✓	✓	✓	
Askalon [29]	A,O,M	71	8	14	14	~			~	
Triana [52]	A,O,S	43	42	7	14	✓	✓		~	
Dryad [8]	I,O,M	64	71	71	43	✓		~	~	

(*) Features: A/I = Academia/Industry, O/N = before/in-or-after 2010, S/M = Single-/Multi-Cluster.
Stages: ✓/~/space () = full/partial/no match.

TABLE III: The top-10 stage with the largest difference in match (%), for systems from academia (A) vs. industry (I).

Field	T11	T10	J2	R2	J7	J3	J6	T9	R3	T8
A	13	31	6	38	38	6	56	13	38	44
I	83	92	50	0	0	42	92	42	67	17

of experiments in Section V). For each group, we first rate the stages in the group (“full”, “partial”, and “no” matches receive a rating of 100%, 50%, and 0%, respectively), then compute an aggregate-percentage as the arithmetic mean of the ratings of each stage in the group.

The results indicate the presence of a wide difference, across schedulers, in the level of specification of both grouped and individual stages. The full specification of stages is rare, and only Apollo, Condor, Dryad, and Borg specify over 50% of their components. Conversely, the most underspecified group is “scheduler management” (less than half of schedulers specify anything about M-components) and “resource management” (no scheduler defines everything, or even three-quarters of the R-components). The most underspecified schedulers include Triana, Sparrow, and Autopilot. Even the four common components selected in Table II are not widely specified across schedulers, and only Condor and Apollo specify them all.

We conclude that the schedulers mapped in this work are on average *underspecified*, and explore in Section V the impact of underspecification on understanding the performance and the operation of schedulers.

D. Academia- vs. Industry-Designed Schedulers

To understand the differences between schedulers designed by academia and by industry, we repeat the mapping process used in the previous section, independently, for the 8 academic and 6 industrial schedulers considered in this work (labeled “A” and “I”, respectively, in column “Feat.”, in Table II).

Figures 5a and 5b depict aggregated results for academia and industry-designed schedulers, respectively. Color-

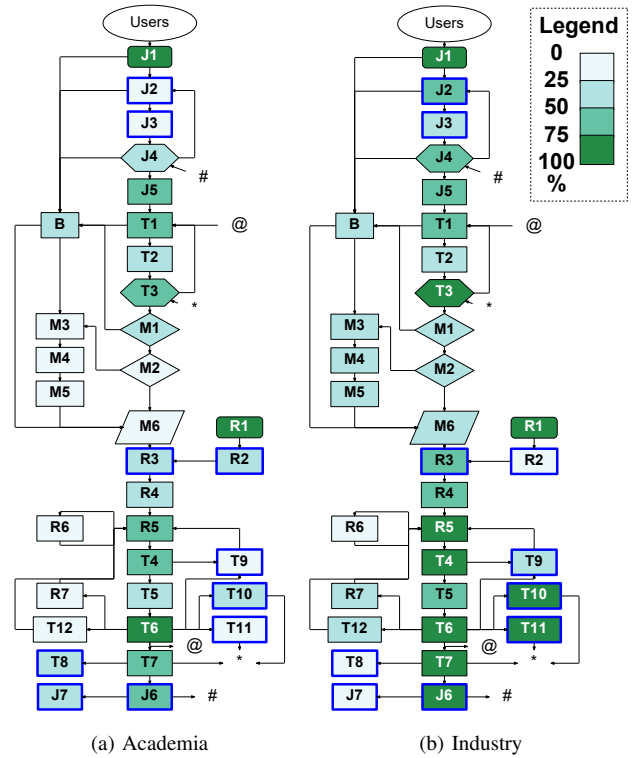


Fig. 5: The specification of stages for academic and industrial schedulers. (Highlighted boxes relate to Table III. Intensities in the heatmap are depicted using a gradient, see Legend.)

gradients depict the aggregated values, using the method used earlier for the group-aggregates included in Table II. The two figures indicate there are some stages where academia and industry underspecify similarly, but many more stages where academia and industry diverge. For example, T1 is defined by [50%,75%) of both academic and industry schedulers, whereas R5 is defined by [50%,75%) of academic schedulers and [75%,100%] (more) of the industry schedulers. Similarly, R6 is rarely specified for all schedulers, indicating resource provisioning (linked to autoscaling, R7) remains understudied.

To quantify differences in underspecification, Table III summarizes the 10 components having the largest absolute difference in specification (*match*), between academia and industry. We observe that task preemption (T10) and recovery (T11) are often specified in work describing industrial systems, yet are underspecified by academia. The industry preference for such features, which have no generic techniques and are thus perceived as engineering-heavy in the academia, can be explained through a simile. An organization needing to select a Linux operating system will need to choose between performance and stability. The organization could choose to take on Linux distributions with higher technical risk, and deploy Fedora or Debian Unstable. These distributions focus on quick and innovative releases, which may improve performance, but may also break more often. Alternatively, the organization could choose for lower risk, installing therefore Red Hat Enterprise Linux (or CentOS), Debian, or an Ubuntu LTS release. They have well-tested, well-engineered features.

TABLE IV: The top-10 stages with the largest difference in match (%), for old, pre-2010 (O) vs. newer (N) schedulers.

Publish	R4	R7	T4	T11	J6	J5	J4	T9	T5	J2
O	21	0	64	29	86	64	64	36	50	14
N	71	36	93	57	57	43	43	14	29	35

Specifying the list of eligible jobs (J2) is also done more often by industry, relatively to the academia. Conversely, filtering based on authorization (R2) and job cleanup (J7) are stages underspecified more by industry than by academia. An alternation of underspecification can occur for stages that from a chain of processing operations, for example, job completion (J6, underspecified less by industry) and job cleanup (J7, underspecified less by academia).

There could be several reasons for these differences in reporting. Correctness of computation (T10) and the consistent state of the system (T11) are perhaps more important for industry, as they have to adhere to (often strict) SLAs, whereas in the academia these aspects may be seen as either a separate concern (thus, they will be mentioned in other, specialized articles) or a technical issue (thus, they will be less specified in peer-reviewed publications). Authorization issues (R2) may not be seen as important when jobs run in a private datacenter or trusted cloud; if this is the case, stage R2 should become more important for industry-designed schedulers, proportionally with the adoption of public-cloud services. Another reason can be practical experience, e.g., job cleanup (J7) must always be implemented following (or as part of) job completion (J6); to a lesser extent, we observe this situation also for tasks (T7 and, notably, T8).

E. Pre-2010 Schedulers vs. Schedulers Since 2010

To understand if there has been a change of focus on scheduling-stages in recent designs, we repeat the mapping process used in Section IV-D, and map in turn the schedulers designed prior to 2010 and since 2010.

Similarly to the previous section, we summarize the largest ten differences in Table IV. From this table, we note a sharp increase in R4, R7, T4, and T11 for schedulers introduced since 2010. We believe this correlates with the increase of dynamicity in real-world environments, leading to increased focus on dynamic resource provisioning and allocation. In contrast, the components J4–6, T9, and T5 show a drop of focus in recent years. This is expected: job completion (J6) and cleanup (J5), and task setup (T5) have used similar techniques for more than a decade, and thus have become tacit knowledge. Interestingly, T9 (task replication) has also been less featured, which could be explained by both standardization and by the realization that the resource waste incurred by large-factor replication is too expensive.

V. EXPERIMENTS WITH THE REFERENCE ARCHITECTURE

In this section, our goal is to understand the impact of stage-underspecification (defined in Section IV) on the performance and operation of real-world scheduling approaches. This gives evidence of the usefulness of the reference architecture (R2).

To achieve our goal, we conduct an experimental evaluation of scheduling approaches, using *dynamic analysis*, that is, analysis based on relevant input traces considered by the scheduling system at runtime. We conduct simulation experiments to evaluate running long-term workloads and real-world experiments to evaluate only the algorithmic (policy) part of the scheduler.

We only summarize in this section the experimental setup and the results. (See technical report [46] for details.)

A. Experiment Setup

1) *Overview*: Table V summarizes the experimental setup. Experiments use an implementation of the reference architecture, in simulation for the experiment in Section V-B, and in a real-world environment for the experiment in Section V-C. The experiments use as input relevant traces collected from real-world environments, and shared by the community through the Grid Workloads Archive [22]. The experiments use a diverse set of metrics to quantify the performance of scheduler workload-management and operation.

2) *Simulation and Real-World Experiments*: For simulations, we extend the community-driven, open-source OpenDC simulation platform [53]. The simulated setup is typical of event-driven simulation of (federated) clusters, and has been implemented similarly by instruments such as CloudSim, GroudSim, etc. We prototype the reference architecture, making configurable the stages T1–2 and R4–5. These stages are constructed based on policies used by real-world schedulers.

For real-world experiments, we run only the abstract parts of the architecture (the algorithms) on a real-world machine, configured as described in Section V-A.5. We subject the algorithms to the incoming load also used during simulations, but for the real-world experiments the scheduler does not follow through with the enforcement of the policy-decisions. This allows the real-world experiments to complete without consuming real datacenter resources, yet to reveal the dynamic behavior of the algorithms under realistic load (in contrast to static, worst-case analysis).

3) *One experiment*: All setups are repeated 32 times, preceded by 4 (discarded) warm-up iterations, to reduce the influence of the underlying platform and cold caches on the measurements. Results, which include hundreds to thousands of samples due to the large input workloads (see *Workloads*), are processed statistically and verified independently.

4) *Workloads*: We experiment here only with workloads of workflows, which are the most complex type of workload supported by the general model. Experimenting with other models, e.g., bags of tasks, while useful, would not have used the full features of workflow-capable schedulers.

We use two traces collected from real-world datacenter-like environments by the community, Askalon [22] and Chronos [54]. Askalon is an engineering workload from a grid cluster that uses workflows to simulate chemical processes. Chronos is an industrial workload from a private cloud that uses workflows to process data collected from an IoT production-environment monitoring industrial equipment.

TABLE V: Experiment configurations.

Sec.	Experiment focus	Workload	Metrics
§V-B	Underspecification	All (§V-A.4)	TRT, JMS, NJSL, JWT
§V-C	Time complexity	All (§V-A.4)	TC

Both workloads have been collected independently from this study, and feature in previous work of the community. Table VI summarizes the characteristics of both workloads: Askalon has more complex workflows, Chronos includes a large job-burst at start.

5) *Datacenter topology*: We consider in this work a datacenter with 32 common-off-the-shelf resources (machines). Half (16) of these machines contain each an Intel i7 (v6) processor, with 4 cores, at a clock rate of 4.1GHz. The other half contain each an Intel i5 (v6) processor, with 2 cores, at a clock rate of 3.5GHz.

For real-world experiments, the scheduler runs on the Intel i7 processor, and Java 8 (1.8.0_162, Oracle engine).

6) *Metrics*: our analysis uses four traditional metrics: (1) Task response time (*TRT*): time elapsed from task submission to task completion, (2) Job makespan (*JMS*): time elapsed from the first task-submission of a job, to the last completion of a task in the job, (3) Normalized Job-Schedule Length [55] (*NJSL*): job makespan normalized by the length of the critical path (the shortest possible execution time of the job), (4) Job waiting time (*JWT*): time elapsed from the first task-submission of a job, to the first start of a task of that job. We also use, for real-world experiments, the metric: (5) Scheduler-stage time-complexity (*TC*): the runtime, or time elapsed since start and until completion, for the stage.

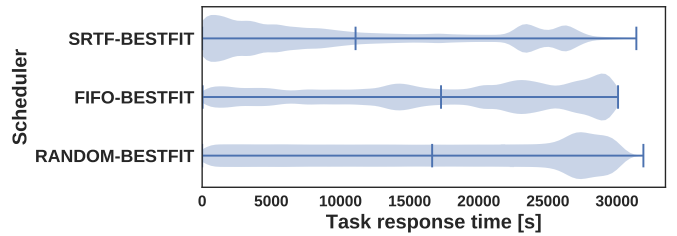
B. Impact of Underspecification on Performance (Simulation)

We investigate the impact of scheduler underspecification on performance. We focus on components that must exist (that is, based on their own publications, the correct operation of the schedulers under study requires these components), yet have been left underspecified by their authors. Although different policies have been shown in the past to lead to different performance, ours is the first study to show the impact of *underspecified* policies across different components of the *same scheduler*. Such underspecification can hamper the reproducibility of a proposed scheduling system. To illustrate this, we use the Borg publication [5] as an example. The publication does not specify its task-sorting, task-allocation, and others policies, although it requires their presence. To highlight the importance of specification, *in this experiment we simulate a Borg-like scheduler, equipped with different policies likely to be used in practice*. We only show here results for Askalon (full results in [46]).

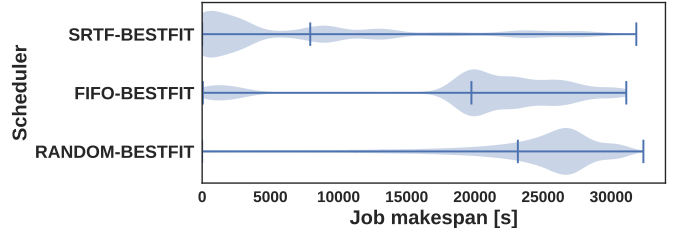
Figure 6a depicts the TRT distribution measured for Borg, when equipped with different sorting and allocation policies.

TABLE VI: Workload characteristics.

Workload	Env.	Application domain	Workflows	Tasks
Askalon	Grid	Engineering, chemistry	757	45,786
Chronos	Cloud	Industrial, IoT	1,024	3,072



(a) TRT distribution per task



(b) JMS distribution per job

Fig. 6: Performance for Askalon. Vertical middle bar is average value of (a) response time, (b) job makespan.

TABLE VII: Average job makespan (JMS), normalized schedule length (NSL), and job waiting time (JWT) per scheduler.

Scheduler	Avg. JMS [s]	Avg. NJSL	Avg. JWT [s]
SRTF-BESTFIT	7,929	5	3,134
FIFO-BESTFIT	19,751	32	2,478
RANDOM-BESTFIT	23,156	206	4,789

Overall, the choice of task-sorting policy (RANDOM, FIFO, or SRTF) is correlated with the distribution of task turnaround times; the allocation policy shows less distinctive results [46]. SRTF gives the best average TRT.

We conduct a similar analysis, per job. Figure 6b depicts the results. The distribution of JMS is more clearly distributed around a limited number of peaks than the distribution of TRT. SRTF-BESTFIT performs significantly better compared to the other two configurations. However, unlike for TRT, for JMS the FIFO policy gives a better average than RANDOM.

Table VII extends the performance analysis with the average NJSL and the average JWT. As for JMS, SRTF-BESTFIT again emerges as best-performing for the NJSL metric. The policy of running the shortest remaining-time first reduces the slowdown, by design.

For JWT, the FIFO-BESTFIT configuration performs best because the FIFO policy for task-sorting preserves the order of submission, which leads to tasks being served with less time spent waiting, on average.

We conclude that *the underspecification of scheduler-stages leads to statistically different performance results*. Even for experts, the results necessarily depend on the configuration (interpretation) of policies for scheduler-stages.

This observation, and the underspecification exemplified in Table II, provide strong motivation for more precise specifications and details of schedulers in scientific literature—this is important in practice, and a new aspect of reproducibility [21].

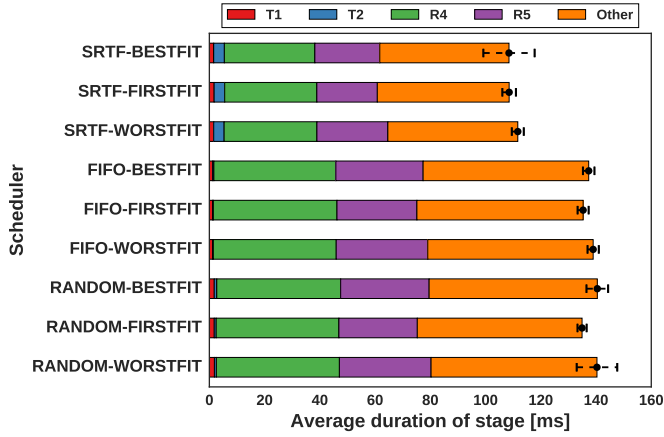


Fig. 7: Time complexity decomposition by stage, for Chronos.

C. Stage-Complexity at Runtime (Real-World Experiment)

We investigate the impact of underspecification of scheduler-stages on the computational complexity (time) of the scheduling process. Unlike traditional, worst-case, asymptotic analysis, we use in this work dynamic analysis: *we implement the scheduling algorithms and run them in real-world experiments, with streaming incoming workload*. Similarly to Section V-B, we explore three policies each for task-sorting (T2) and for resource-allocation (R5), and measure in particular the performance of stages T1–2 and R4–5. We only show here results for Chronos (full results in [46]).

Figure 7 depicts the time-complexity decomposition, by stage, for nine scheduler-configurations. The SRTF task-sorting policy, which delivers the lowest average JMS and NJSL (Section V-B), leads to a significant increase in the time-complexity of the T2 stage, compared to the other task-sorting policies. However, the average overall time complexity of schedulers using SRTF is still the lowest in our experiments, because it produces higher task-throughput and lower TRT (see also Figure 6a); in turn, this reduces the task-queue length, and thus the durations of stages that traverse it, in particular, R4–5.

Similarly to Section V-B, we conclude from these experiments that *the stage-policy can have a significant, non-trivial impact on both single-stage duration and the overall duration of the scheduling iteration*. The non-trivial impact is derived from the *dynamic interplay* between the task throughput, which is likely improved by more compute-intensive policies, and the runtime to get a decision from the scheduler, which is improved by using less compute-intensive policies. The complexity of the scheduling pipeline, which can include policies for each stage in our reference architecture, can further increase the dynamic interplay.

The underspecification of stage-policies may be costly for practitioners, because, when trying to re-use best-practices, or a de facto standard, or a new policy just created elsewhere, underspecification leads to interpretation and guess-work, which in turn can lead to implementing a different scheduler than the original authors did actually evaluate. Similarly, when practitioners create their own alternatives, underspecification could hamper the work of operators and/or QA teams.

VI. RELATED WORK

Our work is the first to: 1) detail the scheduling process for datacenters, 2) consider principles of design for this task, and 3) understand experimentally the impact of underspecification on scheduling performance. (Details in Appendix C.)

This work finds its primary inspiration in Schopf’s multi-stage model of the grid scheduling process [20], subsequent work in the Global Grid Forum (e.g., [56]), and recent surveys [43], [44]. Our work is also related to coarse-grained architectures [57], e.g., for grids [15], clouds [16], and big data systems [17]. Further, this work is related to general reference architectures [58], extending the “Pipes and Filters” model [45].

VII. CONCLUSION AND ONGOING WORK

With cloud operations gaining importance across society, efficient management of datacenter workloads and resources is increasingly critical. The main component responsible for this is the scheduler, which allocates user workloads to resources. Although many scheduling systems already exist, their complex nature and diverse approaches makes it difficult to comprehend and compare them. To address this problem, we propose a reference architecture for datacenter scheduling.

Our reference architecture focuses on a conceptual workflow-based model, whose tasks and flows embody the processes for scheduling in datacenters. We validate this model by mapping to it fourteen existing schedulers, traditional and recent, sourced both from academia and industry. We further give evidence of how the reference architecture can be used in practice, through static and dynamic analysis. Trace-based simulations indicate that the underspecification of components, which is common in published material about schedulers, is problematic for the reproducibility and interpretability of results. From an empirical, trace-based study of stage-complexity at runtime, we find that the separation into stages is meaningful for understanding the dynamic operation of schedulers, and underspecification of key stages can impact significantly online usability. Overall, we conclude that our reference architecture offers a conceptual model that unifies scheduling approaches, is representative for current state-of-the-art, and can help understand datacenter schedulers.

For the future, we plan to explore the scientific and the engineering potential of this architecture: more schedulers, extensions for emerging paradigms (e.g., IoT, edge, and serverless), exploration of the design space. We also foresee our reference architecture to contribute to the design of taxonomies for scheduling systems, enabling further understanding and design studies. (For reproducibility, see Appendix A.)

Long-term, we are currently developing the tools supporting a global competition for scheduling in datacenters, where participants can develop schedulers for various scenarios [53]. We also envision interviews with the original authors of the schedulers used in this work.

ACKNOWLEDGMENTS

Work supported by Dutch projects Vidi MagnaData, Commit.

REFERENCES

- [1] Gens *et al.*, “Worldwide and Regional Public IT Cloud Services 2013–2017 Forecast,” Tech.Rep. by IDC, Doc #242464, Feb 2014.
- [2] Gartner Inc., “Infrastructure and Operations (I&O) Leadership Vision for 2017, section CIO Technology Priorities,” Tech.Rep., 2017.
- [3] Beyer *et al.*, *Site Reliability Engineering: How Google Runs Production Systems*. O’Reilly Media, 2016.
- [4] K. Ousterhout, P. Wendell, M. Zaharia, and I. Stoica, “Sparrow: Distributed, low latency scheduling,” *SOSP*, pp. 69–84, 2013.
- [5] A. Verma, L. Pedrosa, M. Korupolu, D. Oppenheimer, E. Tune, and J. Wilkes, “Large-scale cluster management at Google with Borg,” *EuroSys*, pp. 1–17, 2015.
- [6] T. Tannenbaum, D. Wright, K. Miller, and M. Livny, “Condor: A distributed job scheduler,” *Beowulf Cluster Computing with Linux*, pp. 307–350, 2002.
- [7] E. Boutin, J. Ekanayake, W. Lin, B. Shi, J. Zhou, Z. Qian, M. Wu, and L. Zhou, “Apollo: Scalable and coordinated scheduling for cloud-scale computing,” in *OSDI*, 2014, pp. 285–300.
- [8] M. Isard, M. Budi, Y. Yu, A. Birrell, and D. Fetterly, “Dryad: distributed data-parallel programs from sequential building blocks,” in *EuroSys*, 2007, pp. 59–72.
- [9] D. Thain, T. Tannenbaum, and M. Livny, “Distributed computing in practice: the Condor experience,” *CCPE*, vol. 17, no. 2-4, pp. 323–356, 2005.
- [10] B. Hindman, A. Konwinski, M. Zaharia, A. Ghodsi, A. D. Joseph, R. H. Katz, S. Shenker, and I. Stoica, “Mesos: A platform for fine-grained resource sharing in the data center,” in *NSDI*, 2011.
- [11] Y. Jararweh, M. Al-Ayyoub, A. Darabseh, E. Benkhalifa, M. A. Vouk, and A. Rindos, “Software defined cloud: Survey, system and evaluation,” *FGCS*, vol. 58, pp. 56–74, 2016.
- [12] H. Ghanbari, B. Simmons, M. Litoiu, C. Barna, and G. Iszlai, “Optimal autoscaling in a iaas cloud,” in *ICAC*, 2012, pp. 173–178.
- [13] B. Burns, B. Grant, D. Oppenheimer, E. Brewer, and J. Wilkes, “Borg, omega, and kubernetes,” *Commun. ACM*, vol. 59, no. 5, pp. 50–57, 2016.
- [14] A. Ilyushkin, A. Ali-Eldin, N. Herbst, A. V. Papadopoulos, B. Ghit, D. H. J. Epema, and A. Iosup, “An experimental performance evaluation of autoscaling policies for complex workflows,” in *ICPE*, 2017, pp. 75–86.
- [15] I. T. Foster, C. Kesselman, and S. Tuecke, “The anatomy of the grid: Enabling scalable virtual organizations,” *IJHPCA*, vol. 15, no. 3, pp. 200–222, 2001.
- [16] R. B. Bohn, J. Messina, F. Liu, J. Tong, and J. Mao, “Nist cloud computing reference architecture,” in *SERVICES*, 2011, pp. 594–596, also published as NIST-SP-500-292/2011.
- [17] W. Chang, “NIST big data interoperability framework: Volume 6, reference architecture,” vol. 6, pp. 1–62, 2015. [Online]. Available: https://bigdatawg.nist.gov/_uploadfiles/NIST.SP.1500-6.pdf
- [18] A. Iosup, A. Uta, L. Versluis, G. Andreadis, E. van Eyk, T. Hegeman, S. Talluri, V. van Beek, and L. Toader, “Massivizing computer systems: a vision to understand, design, and engineer computer ecosystems through and beyond modern distributed systems,” in *IEEE ICDCS*, 2018.
- [19] D. Klusáček and S. Tóth, “On interactions among scheduling policies: Finding efficient queue setup using high-resolution simulations,” in *Euro-Par 2014 Parallel Processing - 20th International Conference, Porto, Portugal, August 25-29, 2014. Proceedings*, 2014, pp. 138–149.
- [20] J. M. Schopf, “Ten actions when grid scheduling: The user as a grid scheduler,” *Grid resource management: state of the art and future trends*, pp. 15–23, 2004.
- [21] T. Hoefler and R. Belli, “Scientific benchmarking of parallel computing systems: twelve ways to tell the masses when reporting performance results,” in *SC*, 2015, pp. 73:1–73:12.
- [22] A. Iosup, H. Li, M. Jan, S. Anoep, C. Dumitrescu, L. Wolters, and D. H. J. Epema, “The Grid Workloads Archive,” *Future Generation Comp. Syst.*, vol. 24, no. 7, pp. 672–686, 2008.
- [23] V. Van Beek, J. Donkervliet, T. Hegeman, S. Hugtenburg, and A. Iosup, “Self-expressive management of business-critical workloads in virtualized datacenters,” *Computer*, vol. 48, no. 7, pp. 46–54, 2015.
- [24] I. Altintas, C. Berkley, E. Jaeger, M. Jones, B. Ludascher, and S. Mock, “Kepler: an extensible system for design and execution of scientific workflows,” in *SSDBM*, 2004, pp. 423–424.
- [25] E. Deelman, K. Vahi, M. Rynge, G. Juve, R. Mayani, and R. F. Da Silva, “Pegasus in the cloud: Science automation through workflow technologies,” *IEEE Internet Computing*, vol. 20, no. 1, pp. 70–76, 2016.
- [26] M. C. Kurt, S. Krishnamoorthy, K. Agrawal, and G. Agrawal, “Fault-tolerant dynamic task graph scheduling,” in *SC*, 2014, pp. 719–730.
- [27] T. Hey, S. Tansley, and K. Tolle, *The Fourth Paradigm: Data-Intensive Scientific Discovery*. Microsoft Research, October 2009. [Online]. Available: <https://www.microsoft.com/en-us/research/publication/fourth-paradigm-data-intensive-scientific-discovery/>
- [28] E. Deelman, T. Peterka, I. Altintas, C. D. Carothers, K. K. van Dam, K. Moreland, M. Parashar, L. Ramakrishnan, M. Tauber, and J. S. Vetter, “The future of scientific workflows,” *IJHPCA*, vol. 32, no. 1, pp. 159–175, 2018.
- [29] M. Wicczorek, R. Prodan, and T. Fahringer, “Scheduling of scientific workflows in the askalon grid environment,” *SIGMOD Record*, vol. 34, no. 3, p. 56, 2005.
- [30] N. Russell, W. M. P. van der Aalst, and A. H. M. ter Hofstede, *Workflow Patterns: The Definitive Guide*. MIT Press, 2016.
- [31] F. Liu and J. B. Weissman, “Elastic job bundling: an adaptive resource request strategy for large-scale parallel applications,” in *SC*, 2015, pp. 33:1–33:12.
- [32] M. Comuzzi, C. Kotsokalis, G. Spanoudakis, and R. Yahyapour, “Establishing and monitoring slas in complex service based systems,” in *ICWS*, 2009, pp. 783–790.
- [33] E. Deelman, G. Singh, M. Livny, G. B. Berriman, and J. Good, “The cost of doing science on the cloud: the montage example,” in *SC*, 2008, p. 50.
- [34] K. R. Jackson, K. Muriki, L. Ramakrishnan, K. J. Runge, and R. C. Thomas, “Performance and cost analysis of the supernova factory on the amazon AWS cloud,” *Scientific Programming*, vol. 19, no. 2-3, pp. 107–119, 2011.
- [35] N. R. Herbst, S. Kounev, and R. H. Reussner, “Elasticity in cloud computing: What it is, and what it is not,” in *ICAC*, 2013, pp. 23–27.
- [36] H. Casanova, A. Giersch, A. Legrand, M. Quinson, and F. Suter, “Versatile, scalable, and accurate simulation of distributed applications and platforms,” *J. Parallel Distrib. Comput.*, vol. 74, no. 10, pp. 2899–2917, 2014.
- [37] R. N. Calheiros, R. Ranjan, A. Beloglazov, C. A. F. D. Rose, and R. Buyya, “Cloudsim: a toolkit for modeling and simulation of cloud computing environments and evaluation of resource provisioning algorithms,” *Softw. Pract. Exper.*, vol. 41, no. 1, pp. 23–50, 2011.
- [38] R. Mathá, S. Ristov, and R. Prodan, “Simulation of a workflow execution as a real cloud by adding noise,” *Simulation Modelling Practice and Theory*, vol. 79, pp. 37–53, 2017. [Online]. Available: <https://doi.org/10.1016/j.simpat.2017.09.003>
- [39] E. M. Heien, D. Kondo, A. Gainaru, D. LaPine, B. Kramer, and F. Cappello, “Modeling and tolerating heterogeneous failures in large parallel systems,” in *SC*, 2011, pp. 45:1–45:11.
- [40] B. Javadi, R. K. Thulasiram, and R. Buyya, “Characterizing spot price dynamics in public cloud environments,” *Future Generation Comp. Syst.*, vol. 29, no. 4, pp. 988–999, 2013.
- [41] A. Iosup, O. O. Sonmez, S. Anoep, and D. H. J. Epema, “The performance of bags-of-tasks in large-scale distributed systems,” in *HPDC*, 2008, pp. 97–108.
- [42] P. Pawluk, B. Simmons, M. Smit, M. Litoiu, and S. Mankovski, “Introducing STRATOS: A cloud broker service,” in *IEEE Cloud*, 2012, pp. 891–898.
- [43] S. Singh and I. Chana, “A survey on resource scheduling in cloud computing: Issues and challenges,” *Journal of Grid Computing*, vol. 14, no. 2, pp. 217–264, 2016.
- [44] M. A. Rodriguez and R. Buyya, “A taxonomy and survey on scheduling algorithms for scientific workflows in IaaS cloud computing environments,” *CCPE*, vol. 29, no. 8, 2017, (online pre-print, e4041).
- [45] N. Rozanski and E. Woods, *Software Systems Architecture*. Addison-Wesley Professional, 2005.
- [46] G. Andreadis, L. Versluis, F. Mastenbroek, and A. Iosup, “A reference architecture for datacenter scheduling: Extended technical report,” 2018. [Online]. Available: <https://arxiv.org/abs/1808.04224>
- [47] Z. Zhang, C. Li, Y. Tao, R. Yang, H. Tang, and J. Xu, “Fuxi: A fault-tolerant resource management and job scheduling system at internet scale,” *Proc. VLDB Endow.*, vol. 7, no. 13, pp. 1393–1404, 2014.
- [48] M. Isard, “Autopilot: automatic data center management,” *ACM SIGOPS Operating Systems Review*, vol. 41, no. 2, pp. 60–67, 2007.
- [49] M. Isard, V. Prabhakaran, J. Currey, U. Wieder, K. Talwar, and A. V. Goldberg, “Quincy: fair scheduling for distributed computing clusters,” in *SOSP*, 2009, pp. 261–276.

- [50] S. McGough, L. Young, A. Afzal, S. Newhouse, and J. Darlington, "Workflow enactment in iceni," *UK e-Science All Hands Meeting*, no. August, pp. 894—900, 2004.
- [51] I. Gog, M. Schwarzkopf, A. Gleave, R. N. M. Watson, and S. Hand, "Firmament: Fast, centralized cluster scheduling at scale," in *OSDI*, 2016, pp. 99–115.
- [52] S. Majithia, M. S. Shields, I. J. Taylor, and I. Wang, "Triana: A graphical web service composition and execution toolkit," in *ICWS*, 2004, p. 514.
- [53] A. Iosup, G. Andreadis, V. van Beek, M. Bijman, E. van Eyk, M. Neacsu, L. Overweel, S. Talluri, L. Versluis, and M. Visser, "The OpenDC vision: Towards collaborative datacenter simulation and exploration for everybody," in *ISPDC*, 2017.
- [54] S. Ma, A. Ilyushkin, A. Stegehuis, and A. Iosup, "Ananke: A q-learning-based portfolio scheduler for complex industrial workflows," in *ICAC*, 2017.
- [55] Y.-K. Kwok and I. Ahmad, "Benchmarking the task graph scheduling algorithms," in *IPPS/SPDP 1998*. IEEE, 1998, pp. 531–537.
- [56] C. Grimme, J. Lepping, A. Papaspyrou, P. Wieder, R. Yahyapour, A. Oleksiak, O. Wäldrich, and W. Ziegler, "Towards A standards-based grid scheduling architecture," in *CoreGRID Integration Workshop*, 2008, pp. 147–158.
- [57] ISO, "ISO/IEC/IEEE 42010:2011—Systems and software engineering—Architecture description," International Organization Of Standardization joint report, pp. 1–46, 2011.
- [58] L. Bass, P. Clements, and R. Kazman, *Software Architecture in Practice*. Addison-Wesley Professional, 2003.

APPENDIX A

ARTIFACT DESCRIPTION: “A REFERENCE ARCHITECTURE FOR DATACENTER SCHEDULING: DESIGN, VALIDATION, AND EXPERIMENTS”

A. Abstract

This description contains the information needed to reproduce the experiments outlined in Section V. We have reproduced our work on MacOS El Capitan, Windows 10, and the lightweight Alpine Linux 3.8.

We have released our software as open-source, as part of the SPEC Research community-driven, open-source OpenDC platform [53]. We have also released all the experimental data produced in this work, through the Zenodo Open Science platform¹.

B. Description

1) Check-list (artifact meta information):

- **Algorithm:** Resource management and scheduling system, with policy configurations for stages
- **Program:** Kotlin (JVM-based) codebase and dependencies
- **Compilation:** Java Development Kit, Kotlin, Gradle
- **Data set:** free open-access data collected by the community into the Grid Workload Archive traces [22]
- **Run-time environment:** Java Virtual Machine
- **Output:** Stage execution times and task turnaround times
- **Experiment workflow:** Docker
- **Experiment customization:** Choice of workload, scheduler configurations, and number of repetitions
- **Publicly available?:** Yes, on GitHub and Zenodo

2) *How software can be obtained (if available):* The software can be obtained from the `sc18` release of the OpenDC simulator GitHub repository². We follow the approach of OpenDC and offer next to source-code a container-based deployment approach, based on Docker.

3) *Hardware dependencies:* The software runs on CPU architectures officially supported by Docker and the JVM. We recommend the `x86-64` architecture as other architectures may require building images locally.

4) *Software dependencies:* The software requires Docker to run. The other dependencies are encapsulated within the Docker deployment.

5) *Datasets:* Traces from the Grid Workload Archive [22] are used for the experiments. A copy of the traces is also included in the repository.

C. Installation

The experiments require a running installation of the Docker environment³.

D. Experiment workflow

The experiments take approximately 8 hours to complete. The following files (available in the data archive on Zenodo) need to be present in the current working directory for either of the two experiments to start:

- 1) `/setup.json` - JSON file describing the topology of the datacenter
- 2) `/askalon_workload_ee.gwf` - GWF file containing the trace for the Askalon workload
- 3) `/chronos_exp_noscaler_ca.gwf` - GWF file containing the trace for the Chronos workload

The Askalon experiments can be started as follows:

```
$ docker run -it --rm \
-v $(pwd):/home/gradle/simulator/data \
atlargeresearch/sc18-experiment-runner \
-r 32 -w 4 \
-s data/setup.json \
data/askalon_workload_ee.gwf
```

After the Askalon experiments have been finished, the Chronos experiments can be started in a similar fashion. The result files of any previous experiment runs should be backed up in a different location before running this command, as it will overwrite the result files.

```
$ docker run -it --rm \
-v $(pwd):/home/gradle/simulator/data \
atlargeresearch/sc18-experiment-runner \
-r 32 -w 4 \
-s data/setup.json \
data/chronos_exp_noscaler_ce.gwf
```

For a more in-depth guide we refer to the `README.md` file associated with the software artifacts.

E. Evaluation and expected result

The output can be found as comma-separated values files in the `data/` directory of the repository after the experiments have finished. The data files contain the duration of each stage per scheduling cycle and the metrics of the tasks run during simulation (waiting, execution, turnaround time).

F. Experiment customization

The experiment runner allows for customization of the experiments through the optional command line arguments listed below:

- **-r, --repeat**
The number of times to repeat an experiment for each scheduler.
- **-w, --warm-up**
The number of times to run a warm-up experiment before starting the recorded experiments for each scheduler.
- **-p, --parallelism**
The number of experiments to run in parallel.
- **--schedulers SCHEDULERS**
The list of schedulers to test, separated by spaces.

G. Notes

The amounts of resources (e.g., clusters) in the datacenter can be set in `setup.json`. Each item represents the identifiers of the resource (here, CPU type) to use in the machine. The available CPU types are (1) Intel i7 (4 cores, 4100 MHz) and (2) Intel i5 (2 cores, 3500 MHz).

¹<https://zenodo.org/record/1343629>

²<https://github.com/atlarge-research/opendc-simulator/releases/tag/sc18>

³<https://www.docker.com/>

		Level of Description			Legend
		Well	Poor	None	
Components	Rare	 	 	 	Stakeholder  Non-expert  Expert Reproducible?  Yes  Possibly  No
	Core	 	 	 	

Fig. 8: The matrix depicting the ability of non-expert and expert users to reconstruct components.

APPENDIX B FEASIBILITY OF MAPPING NEW SCHEDULERS

Through a thought experiment, we analyze in this section the feasibility of mapping new datacenter schedulers to our reference architecture. That this is feasible is important, because it illustrates the validity (accuracy) and ease of use of our reference architecture. Conducting real-world experiments requires access to a sufficient fraction of the community’s experts and novices, whose participation in a large-scale study depends on the popularity of the reference architecture—a circular problem we have begun addressing through direct contact, but which falls outside the scope of this work.

To understand the feasibility of mapping, we first define categories for how common the components are, how well the components are described, and for how well trained the analysts are. These categories are: (1) As quantified in Section IV-C, some component are highly present in descriptions of schedulers; without imposing an artificial threshold, we assume some scheduling components are very commonly described in practice and known by all practitioners, and thus constitute *core* components. Conversely, the other components are relatively *rare*, and are not commonly known by practitioners. (2) As described in Section IV-A, components can offer full, partial, and no matches; the matches are derived from these components being well-, poorly, and not described, respectively. (3) We focus on two extreme types of analysts using our reference architecture, either experts or non-experts. We assume *experts* have in-depth knowledge of several schedulers common in datacenters, including both conceptual and technical forms. *Non-experts* are unfamiliar with datacenter scheduling systems, yet are able to reason about and implement systems when provided written source designs and related material e.g., peer-reviewed articles and/or source-code.

We analyze through a thought experiment the full matrix of combinations between component rarity and level of description, for each level of expertise. Figure 8 depicts an overview of the ability of (non-)expert analysts to map (reproduce) a component. We find the following important cases: (1) Well-described components can be reconstructed by both expert and non-expert users, regardless of their rarity; even for the rarest of components, non-experts can simply follow the description.

(2) Core components that are also poorly described can often be reproduced by expert users, which are familiar with the general aspects missing from the description. Non-expert users could reproduce such components through guesswork, largely by looking at similar core components that are well-described. (3) Rare components that are also poorly described could be reconstructed by experts through guessing or sheer experience. However, they cannot be reconstructed by non-experts, who lack knowledge and easily available alternatives cannot reconstruct the missing parts. (4) Finally, components that are not described at all may be reproduced by experts if common, because experts will realize such components *must* be present in a working design and, through sheer experience, devise an exact or alternative design. However, rare components are too specialized or uncommon to be accurately recreated even by experts, unless code or documentation become available, or a conversation with the developers can take place. Moreover, components with no description cannot be reproduced by non-expert users, who normally will not even realize the necessity of such components being present in the design.

APPENDIX C EXTENDED ANALYSIS OF RELATED WORK

In this section, we survey three main classes of related work, in contrast to which we claim significant novelty in Section VI. We explain in the following our claims:

1) *vs. Scheduling architectures for datacenter-based environments*: This work finds its primary inspiration in Schopf’s multi-stage model of the grid scheduling process [20], and subsequent work in the Global Grid Forum (e.g., [56]).

We propose a conceptual model that significantly expands Schopf’s model, with higher granularity of stages, support for advanced concepts typical in modern datacenters (e.g., the concept of workflows, operations such as replication, checkpointing, and migration, etc.) This work also conducts an empirical validation, and trace-based experiments.

In survey work, Rodriguez et al. [44] and Singh et al. [43] discuss the general scheduling process in datacenters and propose taxonomies of resource-management systems. Rodriguez et al. focus on broader architectures, in which schedulers play a coarse-grained, black-box role. Singh et al. cover the resource provisioning and scheduling process, but lack the fine granularity and advanced concepts of this work.

2) *vs. Architectures of grid and cloud computing, and of big data systems*: The architectures proposed at this level are coarse-grained. This holds for various architectures proposed for grids, e.g., in the seminal work of Foster et al. [15] or by the GGF, and the reference architectures proposed by the National Institute of Standards and Technology (NIST) for cloud computing [16] and for big data systems [17]. Our work complements these approaches, and fits within the general framework of the ISO/IEC/IEEE 42010:2011 standard [57], as an “architecture framework” establishing the “common practice” of the field.

3) *vs. Architectures of large-scale software systems*: Our reference architecture aligns with other conceptual models

and architectures for (large-scale) software. As such, it builds upon the existing theory of software architecture design. Rozanski and Woods discuss common principles in software architecture [45] (see Section III for a discussion of how we use their principles). This reference architecture extends the “Pipes and Filters” model [45].

Bass et al. [58] propose a taxonomy of abstractions of software architectures, from which our work matches *reference architectures*—“reference model mapped onto software elements (that cooperatively implement the functionality defined in the reference model) and the data flows between them”.